

CO1AT1-Application - Based Question

NAME: Kalpana J

REGNO: 192521038

COURSE CODE: CSA0613

COURSE NAME: Design and Analysis of Algorithms

15) Application: Multi-Source log Processing System.

Given Recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

This recurrence represents a distributed log Processing System Where:

the inputs logs are divided into two equal halves

Each half is processed recursively
During merging, logarithmic work ($\log n$) is required for searching and synchronization.

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right)$$

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right)\right] + \log n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2\log\left(\frac{n}{2}\right) + \log n$$

$$\log\left(\frac{n}{2}\right) = \log n - 1$$

$$T(n) = 4T\left(\frac{n}{2}\right) + 2(\log n - 1) + \log n$$

$$= 4T\left(\frac{n}{2}\right) + 3\log n - 2$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right)$$

$$T(n) = 4\left[2T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right)\right] + 3\log n - 2$$

$$= 8T\left(\frac{n}{8}\right) + 4(\log n - 2) + 3\log n - 2$$

$$= 8T\left(\frac{n}{8}\right) + 7\log n - 10$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \log\left(\frac{n}{2^i}\right)$$

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log n$$

$$T(n) = nT(1) + \sum_{i=0}^{\log n - 1} 2^i (\log n - i)$$

$$T(n) = n + \sum_{i=0}^{\log n - 1} 2^i (\log n - i)$$

$$T(n) = O(n) + O(n)$$

$$T(n) = O(n)$$

Performance Analysis for large log volumes

- * the log data is recursively divided into two equal parts, allowing parallel processing
- * Although each merge step performs logarithmic work for searching and synchronization.
- * therefore, the running time grows linearly with the number of log records.

4.) Application: Cloud Storage Processing System.

$T(n) = 3T\left(\frac{n}{2}\right) + n$. Using the Master theorem to determine the time complexity.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + n$$

$$a = 3, b = 2, f(n) = n$$

Using the Master theorem, calculate $n^{\log_b a}$

$$n^{\log_b^a} = n^{\log_2^3}$$

$$\log_2^3 \approx 1.585$$

$$n^{\log_2^3} = n^{1.585}$$

$$f(n) = n = n^1$$

$$f(n) = n^1 \neq n^{\log_2^3} = n^{1.585}$$

Since,

$$1 < 1.585$$

$$f(n) = O(n^{\log_2^3 - \epsilon})$$

for some $\epsilon > 0$

$$f(n) = O(n^{\log_b^a - \epsilon})$$

$$T(n) = O(n^{\log_b^a})$$

$$T(n) = O(n^{\log_2^3})$$

Performance Analysis for large cloud storage systems.

- * the cloud storage system divides the input files into 3 smaller subproblems, enabling recursive.
- * the algorithm provides better performance than $O(n^2)$ algorithms and is efficient for processing large-scale storage data.